

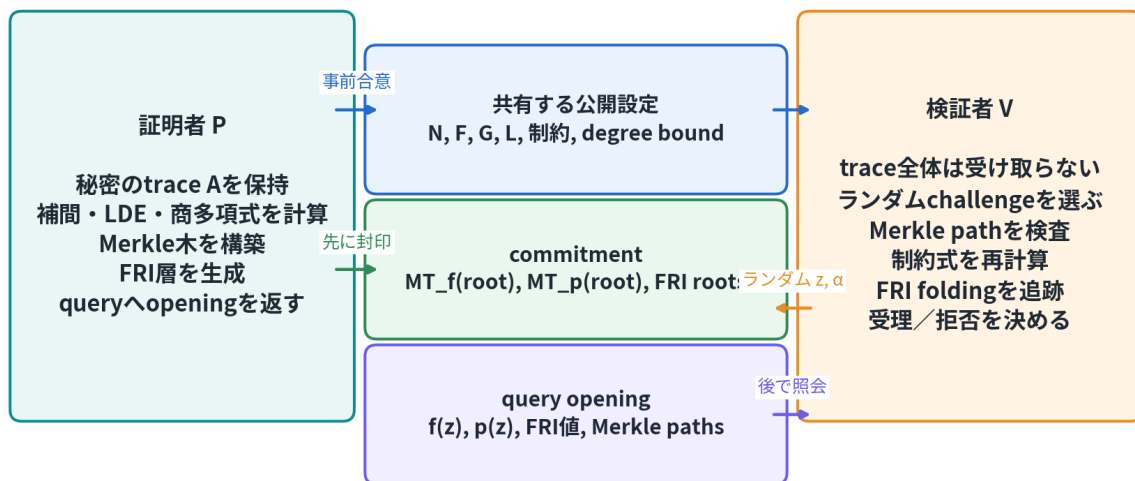
A Walk-Through of a Simple zk-STARK Proof

図解解説書

証明者と検証者の処理を軸に読む

算術化・Merkle commitment・FRI

アクターと証明オブジェクトの全体像



核心：Pは「全データ」を作る。Vは「少数のopening」と「大域構造」を組み合わせて検査する。

原論文：Aleksander Berentsen, Jeremias Lenzi, Remo Nyffenegger (2022)

有限体の基礎説明を省き、プロトコルの役割分担と因果関係に焦点を当てる

はじめに

本書は、Berentsen・Lenzi・Nyffenegger による“A Walk-Through of a Simple zk-STARK Proof”を、証明者と検証者の実際の処理順序に沿って読み解くための補助解説書である。原論文は、長さ 4 の boolean 列を秘匿したまま、その全要素が 0 または 1 であることを証明する一つの対話型 zk-STARK 例を、有限体 F_{17} 上で最後まで数値化している。

原論文の長所は、trace から低次数多項式を作り、制約を商多項式へ変換し、Merkle 木へコミットし、FRI によって低次数性を確認するまでを一つの数値例で追える点にある。一方、教育上の明快さを優先して、現代的な STARK に一般化しない簡略化も明示的に採用している。本書はその構成を尊重し、原論文が説明する内容と、理解のための補足・現代の実装との違いを区別して記述する。

本書の範囲

有限体、剰余演算、群、Lagrange 補間の一般的定義は省略する。必要な式は使用するが、焦点は「なぜその式を作るのか」「証明者と検証者のどちらが何を計算するのか」「不正はどの検査で露見するのか」である。

原論文の立場

原論文は対話型プロトコルを記述する。非対話型化については Fiat-Shamir heuristic により検証者のランダム challenge を transcript のハッシュから導出できると説明している。本書もまず対話型の因果関係を説明し、その後に非対話型との対応を示す。

原論文の構成と本解説書の読み方

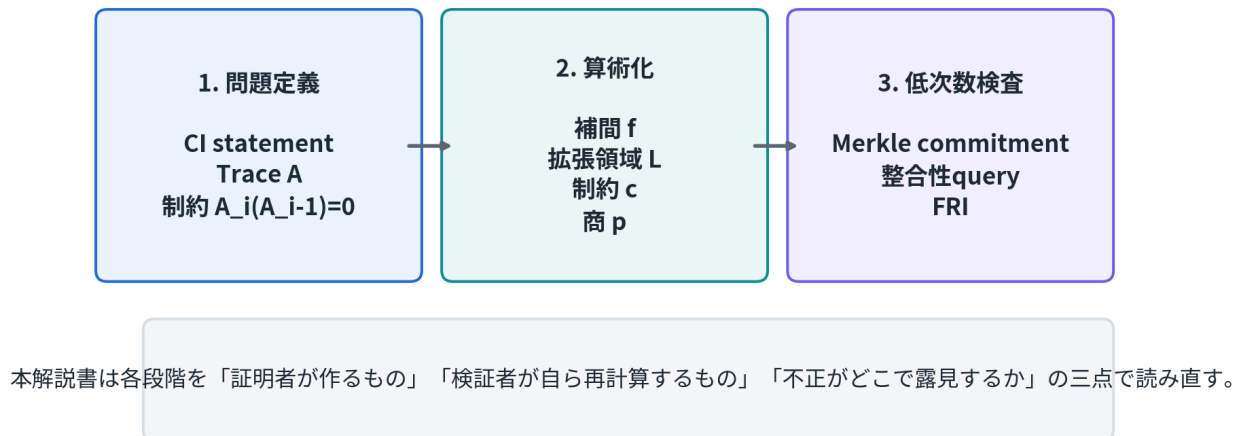


図 1 原論文の三部構成と本書の読み方

本書を読む際の三つの視点

1. 証明者は、秘密の trace 全体を使って、評価表・商多項式・Merkle 木・FRI 層を生成する。
2. 検証者は、trace 全体を受け取らず、公開設定、commitment root、少数の opening から必要な関係だけを自ら再計算する。
3. Merkle、制約等式、FRI は別の役割を持つ。Merkle は値の事後変更を防ぎ、制約等式は計算の意味を結び付け、FRI は低次数性を検査する。

証明者と検証者の責任分界

フェーズ	証明者 P	検証者 V
算術化	traceからf,c,pを構成	degree boundと制約式を把握
commitment	全評価表をMerkle木へ封印	rootを保持し、後のopeningと照合
整合性query	指定点の値とpathを返す	$p \cdot u = f^2 - f$ を自分で再計算
FRI commit	各fold層を計算しrootを送る	各root後にランダム α を選ぶ
FRI query	対点値とpathを返す	folding式を経路ごとに再計算
判定	証明データを完成	一つでも不一致なら拒否

VはPの計算を全面的に再実行しない。Pが先に固定したデータの「少数の局所関係」を再計算する。

図 2 証明者と検証者の責任分界

目次

本書は原論文の構成を保ちながら、証明者と検証者の処理順序に沿って再編している。

表 1 章構成

章	内容
第 1 章	原論文が証明する命題とアクター
第 2 章	最初から最後まででの証明シーケンス
第 3 章	trace から低次数多項式 f への変換
第 4 章	拡張評価領域 L と Reed-Solomon 冗長性
第 5 章	制約多項式 c と composition polynomial p
第 6 章	Merkle commitment と整合性 query
第 7 章	低次数検査が必要となる理由
第 8 章	FRI commitment round
第 9 章	FRI querying round
第 10 章	偽の trace に対する検出
第 11 章	証明者と検証者の責任分界
第 12 章	Fiat-Shamir による非対話化
第 13 章	現代的 ZK-STARK との対応
第 14 章	検証者の最終判定
Appendix	数値表・式の逆引き・用語集・参考文献

第1章 原論文が証明する命題とアクター

原論文参照 : pp. 2-4, 35-36

1.1 証明対象は「秘密列が boolean であること」である

原論文の computational integrity statement は、「証明者が長さ N の整数列 A を持ち、その全要素 A_i が 0 または 1 である」というものである。数値例では $N=4$ とし、正しい trace を $A_{\text{true}}=[1,0,1,1]$ 、偽の trace を $A_{\text{false}}=[2,0,1,1]$ と置く。証明者は A そのものを検証者へ開示せず、列の全要素が boolean であることを納得させる。

$$A_i(A_i - 1) = A_i^2 - A_i = 0 \quad \text{for } i=0, \dots, N-1 \quad (1)$$

この制約は、 $A_i=0$ または 1 のときに限って 0 になる。したがって、全行について式(1)が成立すれば、CI statement が成立する。この例には隣接行の遷移規則はなく、各セルへの同一の boolean 制約だけが存在する。これは一般的な AIR より単純であり、原論文自身も複雑な問題へそのまま一般化しないと断っている。

証明者の目的

A を開示せずに、 A の全セルが制約を満たすことを、一つの低次数構造と少数の query へ変換する。

検証者の目的

A の全セルを読む代わりに、commitment・制約等式・低次数性を組み合わせ、偽の A を用いた証明が高確率で失敗することを確認する。

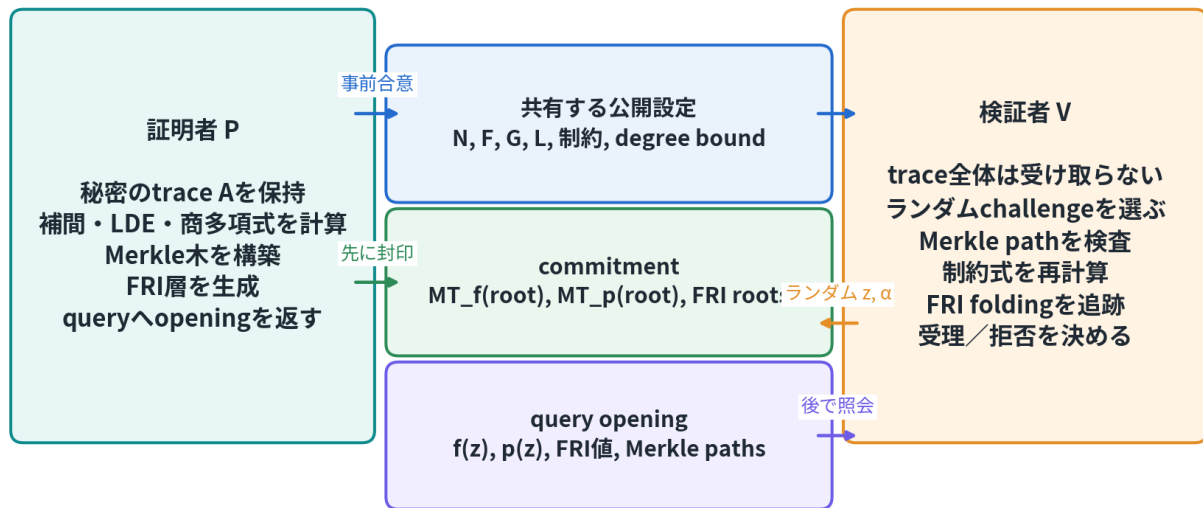
1.2 両者が共有する公開設定

証明開始前に、証明者と検証者は、 N 、有限体 F 、trace 領域 G 、拡張評価領域 L 、boolean 制約、許容次数上限、Merkle hash の仕様、FRI のラウンド数などを共有する。数値例では $F=F_{1_7}$ 、 $G=\{1,4,13,16\}$ 、 $L=F_{1_7} \setminus \{0\}$ である。

表2 情報の公開・秘密境界

区分	内容	誰が知るか
秘密	trace A	証明者のみ
公開 statement	A の全要素が 0 または 1	双方
公開設定	N, F, G, L , 制約, degree bound	双方
証明メッセージ	Merkle roots, opening values, paths, FRI roots, final constant	検証者へ送付

アクターと証明オブジェクトの全体像



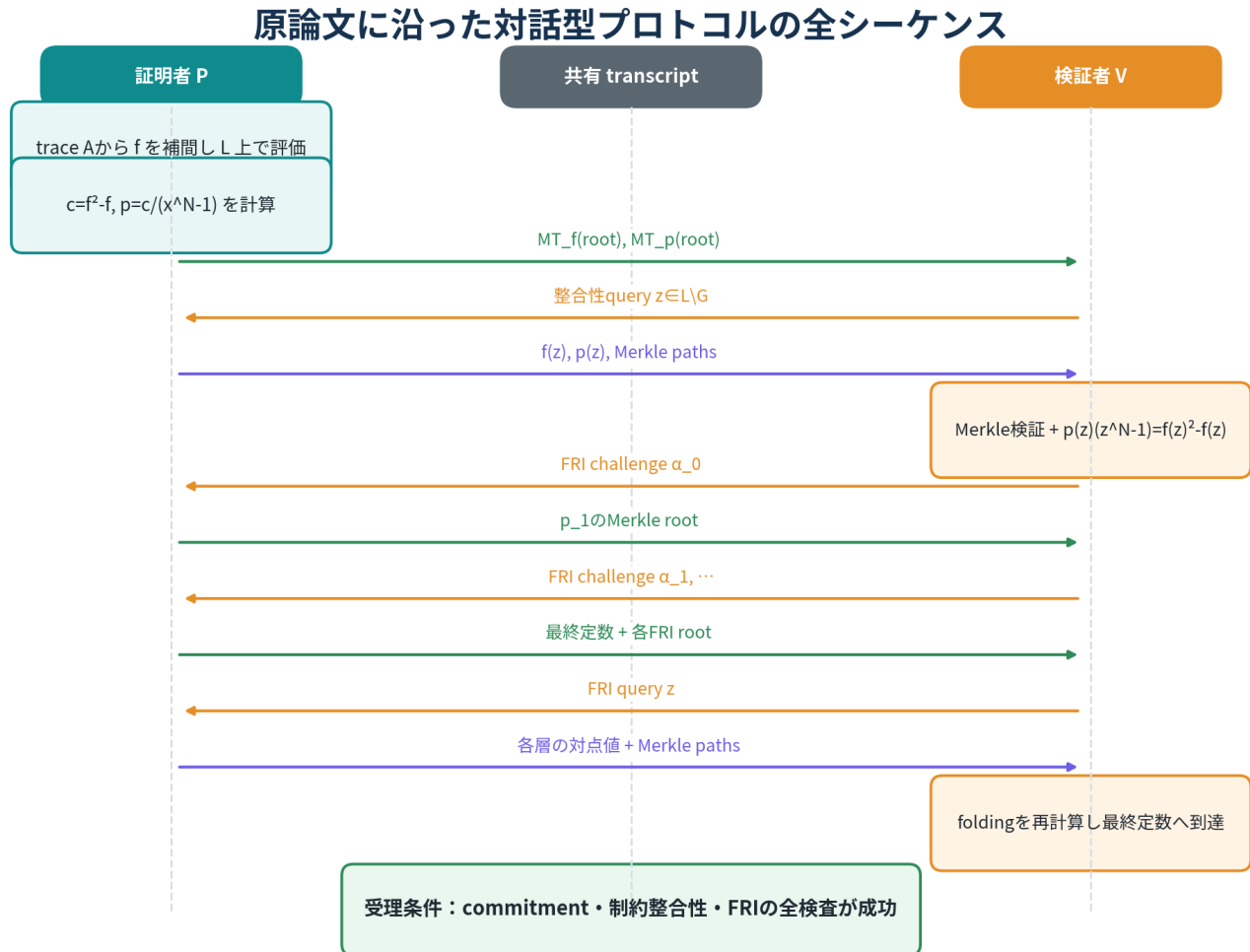
核心：Pは「全データ」を作る。Vは「少数のopening」と「大域構造」を組み合わせで検査する。

図3 アクターと証明オブジェクトの全体像

第2章 最初から最後まででの証明シーケンス

原論文参照：pp. 22-23, 27-33, 35-36

原論文のプロトコルは、大きく「算術化」「制約との整合性 query」「FRI commitment round」「FRI querying round」の四つに分かれる。証明者はまず trace 全体から多項式と評価表を構成し、検証者が query を選ぶ前に Merkle root で封印する。その後、検証者はランダム点を選び、証明者が返した少数の値について、Merkle membership と代数的関係を自ら確認する。



2.1 十段階で見る全体フロー

4. P と V は CI statement、制約、F、G、L、次数上限を共有する。
5. P は trace A を評価値とする次数 $N-1$ 以下の多項式 f を補間する。
6. P は f を拡張領域 L で評価し、Reed-Solomon 型の冗長な評価表を得る。
7. P は $c=f^2-f$ を作り、 G を根にもつ $u=x^N-1$ で割って $p=c/u$ を構成する。
8. P は f と p の評価表を二つの Merkle 木へコミットし、root を V へ送る。
9. V はランダムな z を選び、P から $f(z)$ 、 $p(z)$ 、Merkle paths を受け取る。
10. V は root を再構成し、 $p(z)(z^N-1)=f(z)^2-f(z)$ を自分で確認する。
11. P と V は FRI commitment round を行い、 p を段階的に fold し、各層へコミットする。

12. V は FRI query 点を選び、各層の対点 opening を受け取り、folding 式を最終定数まで再計算する。
13. V はすべての Merkle 検証、制約整合性、FRI 検査が成功した場合だけ証明を受理する。

順序が安全性を作る

P は必ず先に commit する。V のランダム challenge は commitment の後に決まる。この前後関係により、P は query された場所だけを後から正しく作り直せない。

第3章 証明者は trace を多項式 f へ変換する

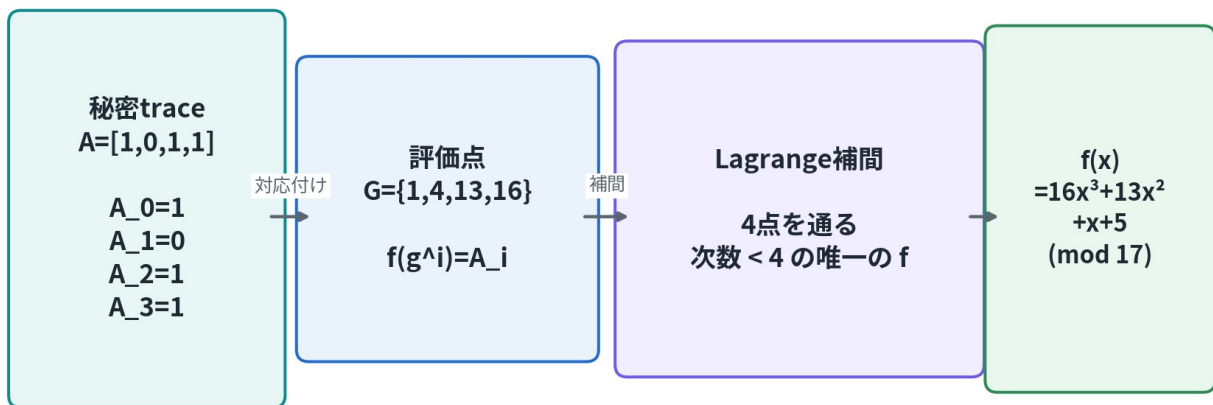
原論文参照：pp. 8-13

3.1 trace の各位置を G 上の評価点へ割り当てる

証明者は、 G の generator g を使い、trace の第 i 要素を評価点 g^i へ対応させる。数値例では $g=4$ 、 $G=\{1,4,13,16\}$ であり、 $f(1)=1$ 、 $f(4)=0$ 、 $f(13)=1$ 、 $f(16)=1$ を満たす次数 4 未満の多項式 f を求める。

$$f(g^i) = A_i \quad \text{for } i=0, \dots, N-1 \quad (2)$$

trace A を低次数多項式 f に置き換える



V は trace を見ない。ただし「 f の degree bound」と後続の opening を検査する。

図5 trace A を評価点 G へ割り当て、低次数多項式 f を補間する

3.2 なぜ「低次数の」補間多項式を選ぶのか

N 個の異なる評価点に対して、次数 $N-1$ 以下の補間多項式は一意である。高次数の多項式も同じ N 点を通り得るが、それを許すと証明者が拡張領域上の値を都合よく成形できる自由度が増える。したがって、trace を表す標準的な符号化として、次数 $N-1$ 以下の唯一の f を採用する。

$$f(x) = -x^3 - 4x^2 + x + 5 = 16x^3 + 13x^2 + x + 5 \pmod{17} \quad (3)$$

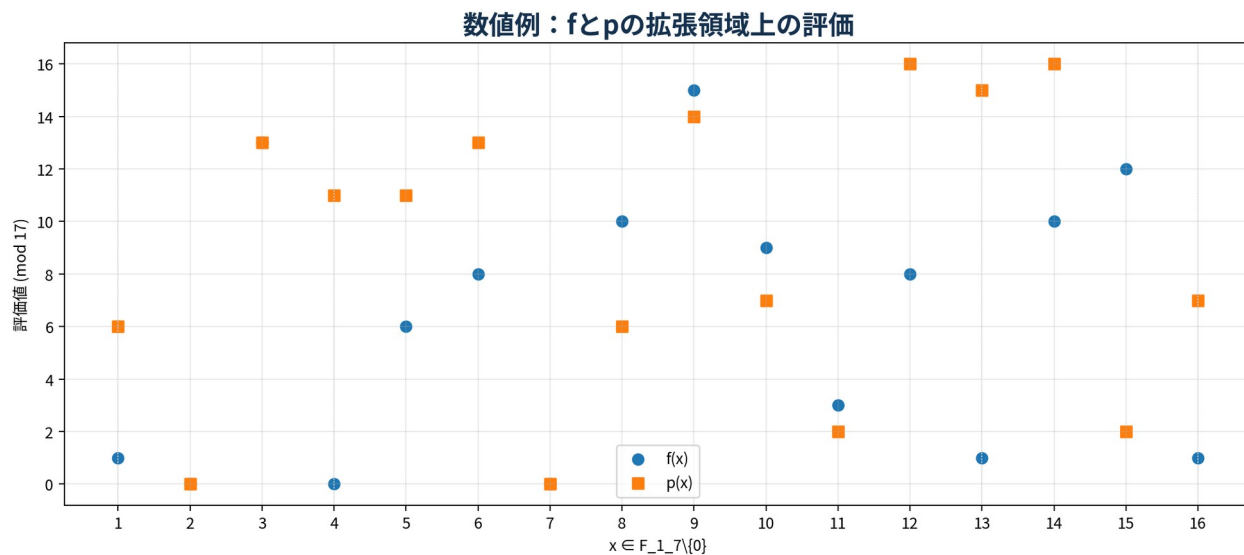
この式は証明者が計算する。検証者は係数を受け取って f を再構成するわけではない。後で、 f の評価値が commitment に属することと、制約商 p との関係が query 点で成立することを検査する。

証明者が保持するもの

f の係数または L 上の全評価値、trace A 、後に Merkle 木を作るための全 leaf を保持する。

検証者が保持するもの

f の degree bound と Merkle root だけでよい。query 時に少数の $f(z)$ を受け取り、root と照合する。

図6 数値例における $f(x)$ と $p(x)$ の拡張領域上の評価

第4章 証明者は f を大きな領域 L へ拡張する

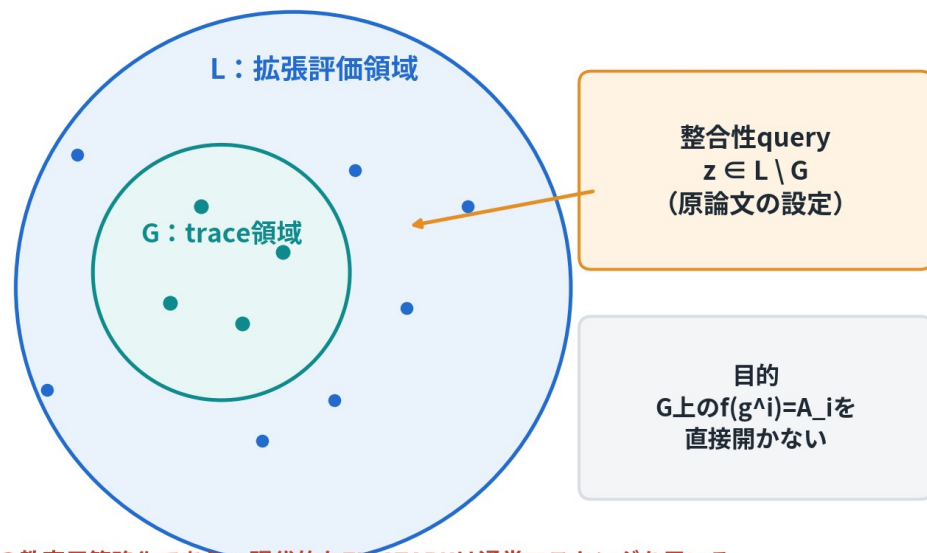
原論文参照：pp. 11-13, 20-22

4.1 拡張評価は冗長性を作る

証明者は f を trace 領域 G だけでなく、より大きな領域 L で評価する。原論文はこれを Reed-Solomon error correction code の生成として説明する。 f の次数上限が固定されているため、 L 上の多数の値は互いに独立ではなく、少数の自由度から決まる冗長な評価表になる。

この冗長性には二つの意味がある。第一に、異なる低次数多項式同士が一致できる点数は次数で制限される。第二に、 L を大きくすると、一致点数を増やさずに異なる点数を増やせるため、ランダム query で偽の多項式を引き当てる確率が高くなる。

G・L・query領域の関係（原論文の簡略化）



注意：これは論文の教育用簡略化であり、現代的なZK-STARKは通常マスキングを用いる。

図7 trace 領域 G 、拡張領域 L 、原論文における query 領域

4.2 「一つの嘘が広がる」の正確な意味

trace の一セルを変更した後に低次数補間をやり直すと、元の f とは別の低次数多項式が得られ、その評価は L 上の多数の点で変化する。ただし、変更後の評価表も一つの正規な低次数符号語であり、FRI だけなら通過し得る。変更後の trace が boolean 制約を破ることは、 $c=f^2-f$ と p の整合性検査によって拒否される。

一方、LDE 後の評価表の一値だけを改ざんして Merkle 木へ入れると、その表は一般にどの低次数多項式の評価表にも近くなる。この種類の不正は FRI が検査する。すなわち、AIR・制約商と FRI は異なる種類の嘘を受け持つ。

重要な区別

FRI は「計算が正しい」ことを直接検査しない。FRI が検査するのは低次数構造である。計算の意味は c と p の制約等式が与える。

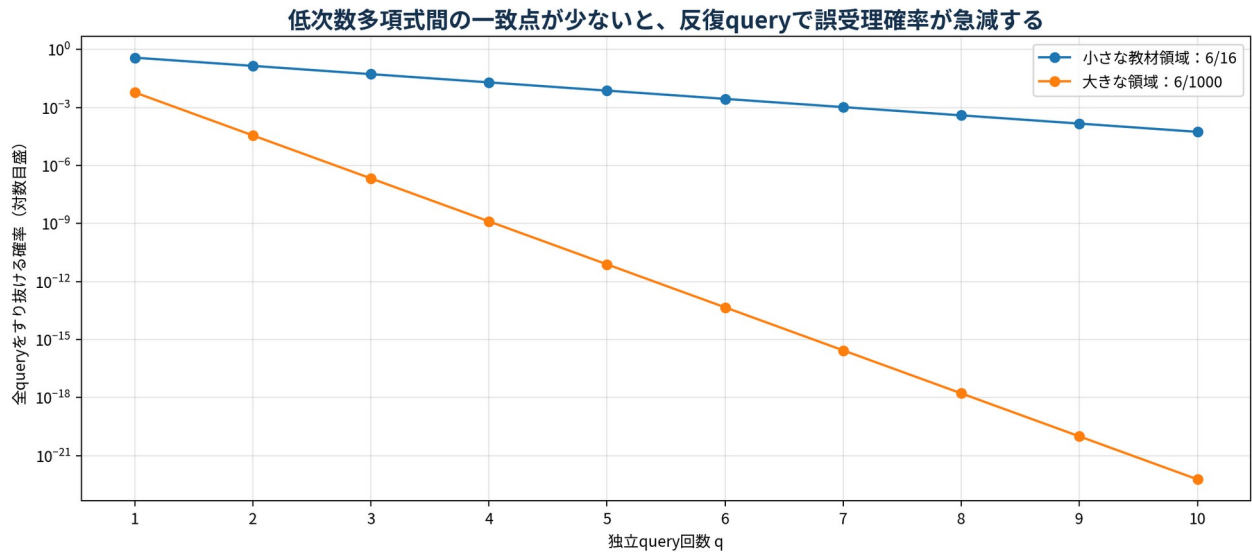


図 8 query 回数と偽証明が全 query をすり抜ける確率の関係

原論文 p.20 の数値について

原論文は、1000 点中 6 点でしか一致しない場合の 1 回 query の検出確率を 94.4%と記載する。しかし、本文中の 6/1000 という比率に従えば、見逃し確率は 0.6%、検出確率は 99.4%である。本書では式に基づく 99.4%を採用し、原文の 94.4%は数値上の誤記とみなす。

第5章 証明者は制約を c と p へ変換する

原論文参照：pp. 13-16

5.1 boolean 制約を constraint polynomial へ移す

証明者は、trace 上の制約 $A_i^2 - A_i = 0 \wedge f(g^i) = A_i$ を代入し、 L 全体で定義される constraint polynomial c を作る。

$$c(x) = f(x)^2 - f(x) \quad (4)$$

正しい trace であれば、 G の各点で $f(g^i)$ は 0 または 1 であるため、 $c(g^i) = 0$ となる。つまり、 G の全要素が c の根になる。ここで、制約の「全行での成立」が、多項式の「所定の根集合を持つこと」へ置き換わる。

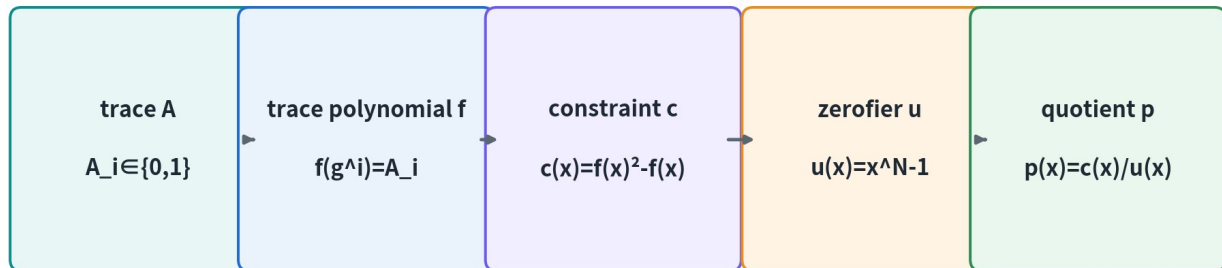
5.2 G の消失多項式で割る

G は位数 N の乗法部分群であるため、 G の全点を根にもつ多項式は $u(x) = x^N - 1$ と簡潔に書ける。証明者は c を u で割り、composition polynomial p を作る。

$$u(x) = \prod_{i=0}^{N-1} (x - g^i) = x^N - 1 \quad (8)$$

$$p(x) = c(x)/u(x) = (f(x)^2 - f(x))/(x^N - 1) \quad (9)$$

算術化の因果関係： $A \rightarrow f \rightarrow c \rightarrow p$

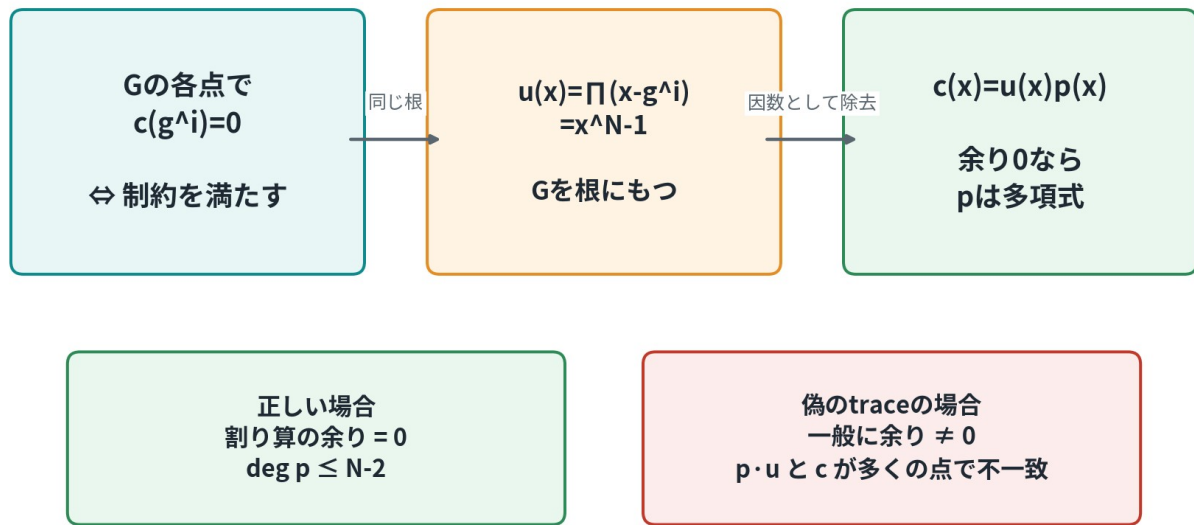


正しい trace なら c は G の全点で $0 \rightarrow u$ で割り切れる $\rightarrow p$ は低次数多項式になる

$\forall i$ は「 p が低次数であること」と「 $p \cdot u = c = f^2 - f$ 」を別々に検査する。

図9 算術化の因果関係： A から f 、 c 、 u 、 p へ

なぜ「割り切れること」が計算の正しさに対応するのか

図 10 G 上で制約が成立することと、 u で割り切れることの対応

5.3 数値例の composition polynomial

$$p(x) = x^2 + 8x - 3 \pmod{17}, \quad \deg p = 2 = N-2$$

原論文の toy example では制約が一種類だけなので、この商 p 自体を composition polynomial と呼ぶ。原論文の脚注が述べるとおり、一般の STARK では複数の制約から複数の商を作り、検証者由来のランダム係数による線形結合を composition polynomial とする。

現代的 STARK への対応

本例の p は「単一制約の商」である。一般的な AIR では、境界制約、遷移制約、selector、複数列の関係などを個別に商へ変換し、ランダムに束ねる。

第6章 証明者は f と p へコミットし、検証者は整合性を点検する

原論文参照：pp. 17-23

6.1 なぜ先に Merkle root を送るのか

検証者が query するたびに、証明者がその場で都合のよい $f(z)$ と $p(z)$ を返せるなら、式を満たす値を毎回捏造できる。そこで証明者は、 L 上の f 評価表と p 評価表をそれぞれ Merkle 木へ入れ、二つの root を先に送る。

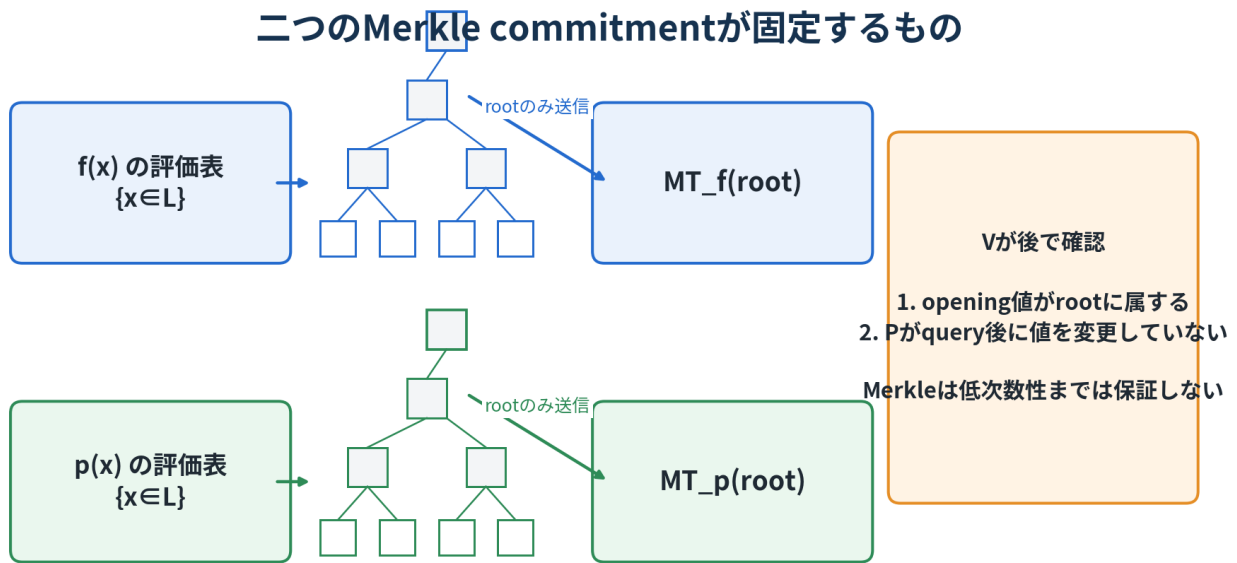


図 11 f と p の評価表に対する二つの Merkle commitment

Merkle root は「値を先に固定した」ことを保証するが、その評価表が低次数多項式から来たことまでは保証しない。そのため、後に FRI が必要となる。

6.2 検証者が query 点で行う三つの検査

検証者は $z \in L \setminus G$ をランダムに選び、証明者へ送る。証明者は $f(z)$ 、 $p(z)$ 、それぞれの Merkle path を返す。検証者は、第一に $f(z)$ と $p(z)$ が先に受け取った root へ到達するかを確認する。第二に、公開情報 N と z から $u(z) = z^{N-1}$ を自分で計算する。第三に、次の等式を確認する。

$$p(z)(z^N - 1) \stackrel{?}{=} f(z)^2 - f(z) \quad (10)$$

整合性query：Vが一つの点で何を再計算するか

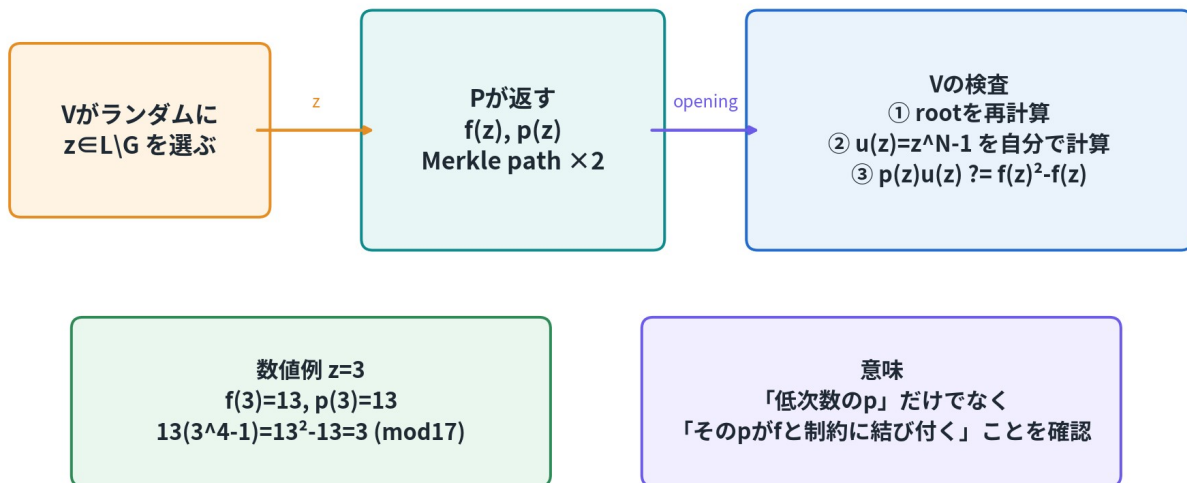


図 12 整合性 query で検証者が再計算する内容

6.3 数値例 $z=3$

原論文では、検証者が $z=3$ を選び、証明者が $f(3)=13, p(3)=13$ と二本の Merkle path を返す。検証者は path から root を再構成した後、有限体 F_{17} 上で等式を確認する。

$$13 \cdot (3^4 - 1) = 13^2 - 13 = 3 \pmod{17}$$

検証者が信用しないもの

V は P が「 $c(3)=3$ 」と言うことを信用しない。 $f(3)$ から $f(3)^2 - f(3)$ を自分で計算し、 $p(3)$ と公開 $u(3)$ から得た左辺と比較する。

第7章 なぜ低次数検査が不可欠なのか

原論文参照：pp. 19-26

7.1 別の低次数 p を選ぶ攻撃

証明者が正しい f に対して無関係な低次数 p_{false} をコミットした場合、 $p_{\text{false}} \cdot u$ と $f^2 - f$ は異なる低次数多項式になる。二つの次数 d 以下の多項式は高々 d 点でしか一致しないため、 L が十分大きければランダム query で不一致を発見しやすい。

7.2 高次数を許すと局所 query に合わせられる

しかし、 f と p に高次数を許すと、評価領域の大部分で式(10)を満たし、偽の trace 位置だけで破るように多項式を成形できる。原論文は次数 15 の f^H と p^H を例示し、 L のほぼ全点で式が成立する攻撃を示す。この場合、整合性 query だけでは偽証明を高確率で受理してしまう。

なぜ整合性 query だけでは足りないのか

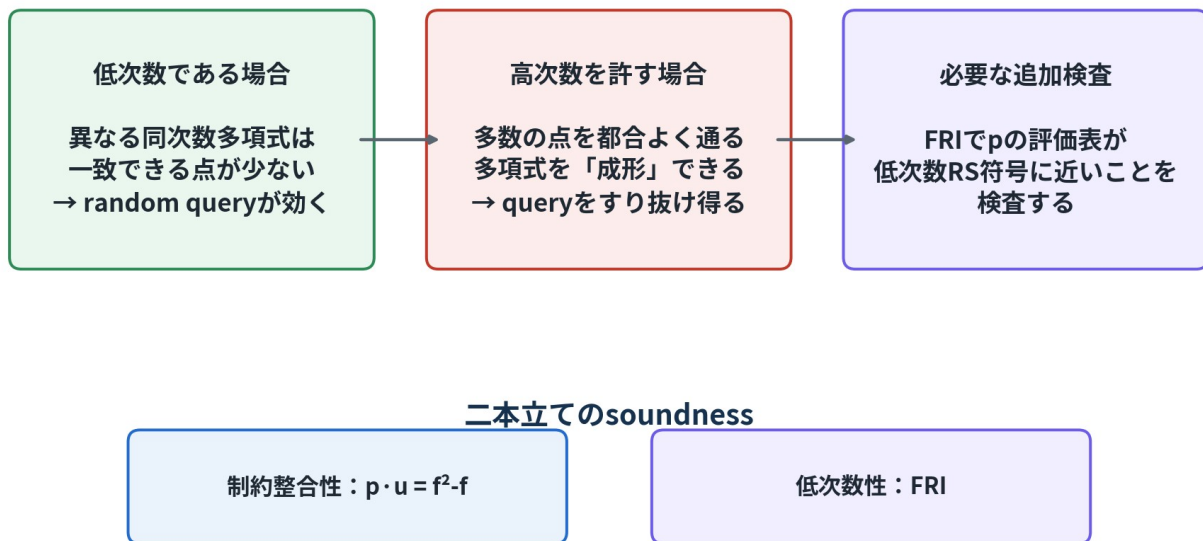


図 13 整合性 query と低次数検査は別々に必要である

二つの検査の関係

制約 query は「 p が f と制約に結び付くか」を確認する。FRI は「 p の評価表が本当に低次数構造を持つか」を確認する。どちらか一方だけでは不十分である。

第8章 FRI commitment round：証明者が層を作る

原論文参照：pp. 26-30

8.1 even/odd 分解と degree halving

FRI は、 p を偶数次数部分と奇数次数部分へ分け、検証者が選ぶランダム係数 α で線形結合する。

$$p_k(x) = g_k(x^2) + x h_k(x^2) \quad (11)$$

$$p_{k+1}(y) = g_k(y) + \alpha_k h_k(y) \quad (12)$$

p_k の次数が d であれば、 g_k と h_k の次数は概ね $d/2$ になる。したがって p_{k+1} も概ね半分の次数になる。評価領域も L から L^2 、 L^4 へと半分ずつ縮む。

FRI commitment round：次数と領域を段階的に半分へ

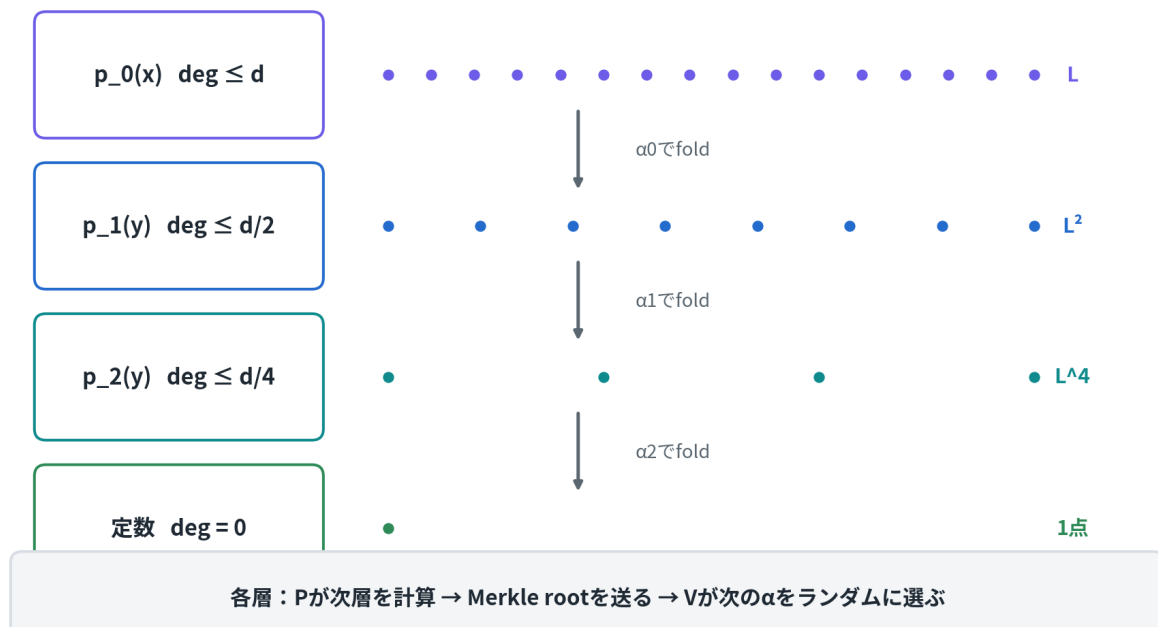


図 14 FRI commitment round で次数と評価領域を縮小する

8.2 各ラウンドの役割分担

証明者は p_k の全評価表から p_{k+1} の全評価表を計算し、Merkle root を検証者へ送る。検証者はその root を受け取った後に次のランダム係数 α_k を選ぶ。root が先、 α が後という順序により、証明者は α に応じて前層を作り直せない。

証明者の仕事

各層の全評価値を計算し、Merkle 木を構築する。計算量とメモリの大部分は証明者側にある。

検証者の仕事

commitment round では全評価値を受け取らず、各層の root と最終定数を保持する。正しい folding は query round で少数経路について確認する。

8.3 数値例

数値例： $p(x)=x^2+8x-3$ を二回foldする

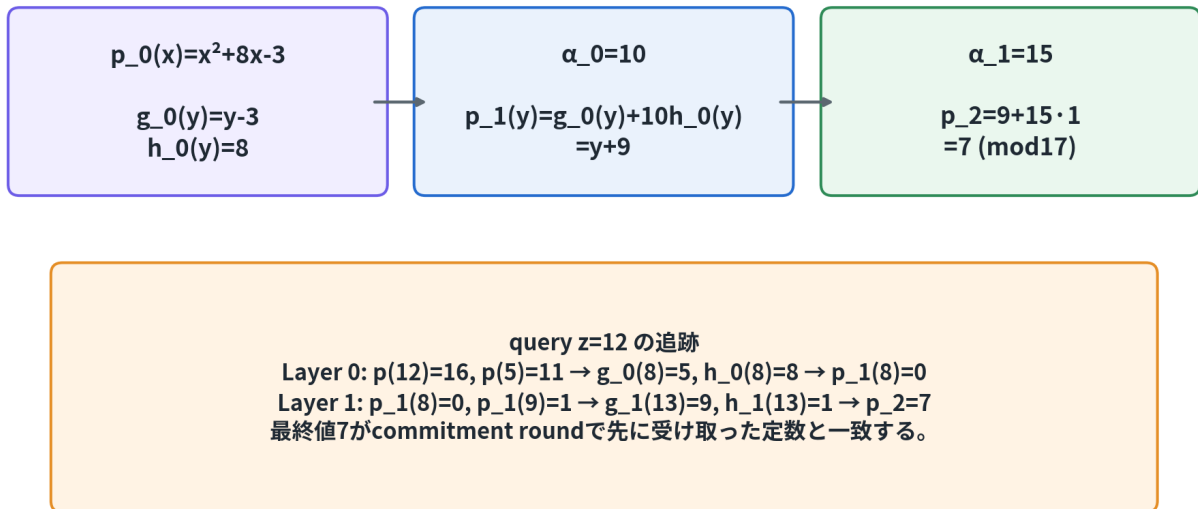


図 15 $p(x)=x^2+8x-3$ を二回 fold して定数 7 へ到達する

第1ラウンドでは $g_0(y)=y-3$ 、 $h_0(y)=8$ であり、 $\alpha_0=10$ から $p_1(y)=y+9$ を得る。第2ラウンドでは $g_1=9$ 、 $h_1=1$ であり、 $\alpha_1=15$ から $p_2=7$ を得る。証明者は p_1 の Merkle root と最終定数 7 を検証者へ送る。

第9章 FRI querying round：検証者が一本の経路を再計算する

原論文参照：pp. 30-33

9.1 なぜ $p(z)$ と $p(-z)$ の二点を開くのか

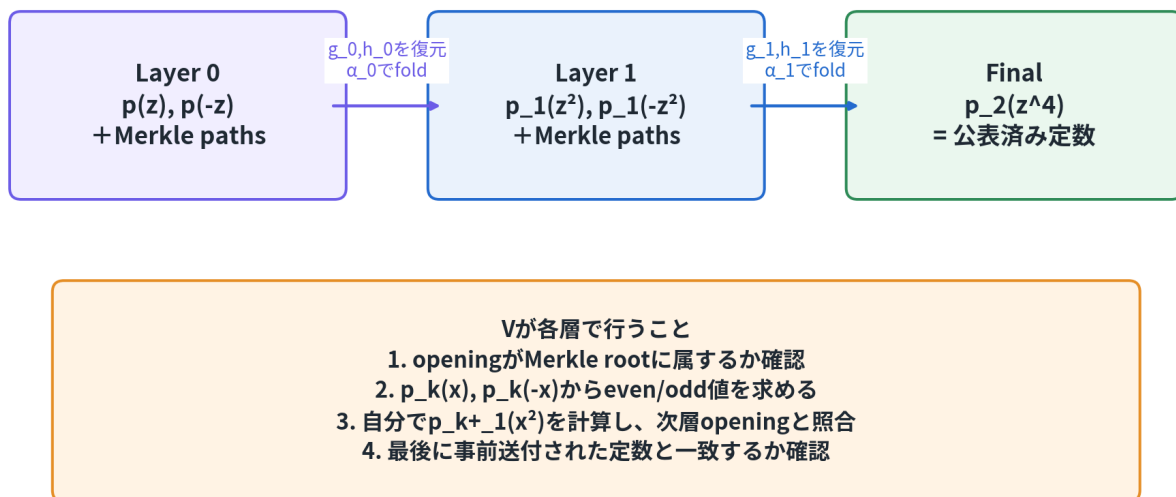
$p(z)=g(z^2)+zh(z^2)$ 、 $p(-z)=g(z^2)-zh(z^2)$ であるため、この二式から $g(z^2)$ と $h(z^2)$ を求められる。検証者は得られた g と h に commitment round で選んだ α を代入し、次層の期待値 $p_1(z^2)$ を自ら計算する。

$$g(z^2) = (p(z)+p(-z))/2$$

$$h(z^2) = (p(z)-p(-z))/(2z)$$

$$p_1(z^2) = g(z^2) + \alpha_0 h(z^2)$$

FRI querying round：一本のランダム経路を最終定数まで追う



Pが層を作る。Vは層を作り直さず、選ばれた経路について局所的なfolding式を再計算する。

図 16 FRI query が各層を貫き、最終定数へ到達する流れ

9.2 Merkle opening との組合せ

証明者は、各層の必要な二点値と Merkle path を返す。検証者は値が各層の root に属することを確認した上で、folding 式から次層の期待値を計算し、次層の opening と一致するかを確認する。この処理を最終層まで繰り返す。

9.3 数値例 $z=12$

原論文の数値例では、 $p(12)=16$ 、 $p(-12)=p(5)=11$ である。検証者はこれらから $g_0(8)=5$ 、 $h_0(8)=8$ を得て、 $\alpha_0=10$ から $p_1(8)=0$ を計算する。次に $p_1(8)=0$ 、 $p_1(9)=1$ から $g_1(13)=9$ 、 $h_1(13)=1$ を得て、 $\alpha_1=15$ から $p_2=7$ を計算する。これは commitment round で事前に受け取った定数 7 と一致する。

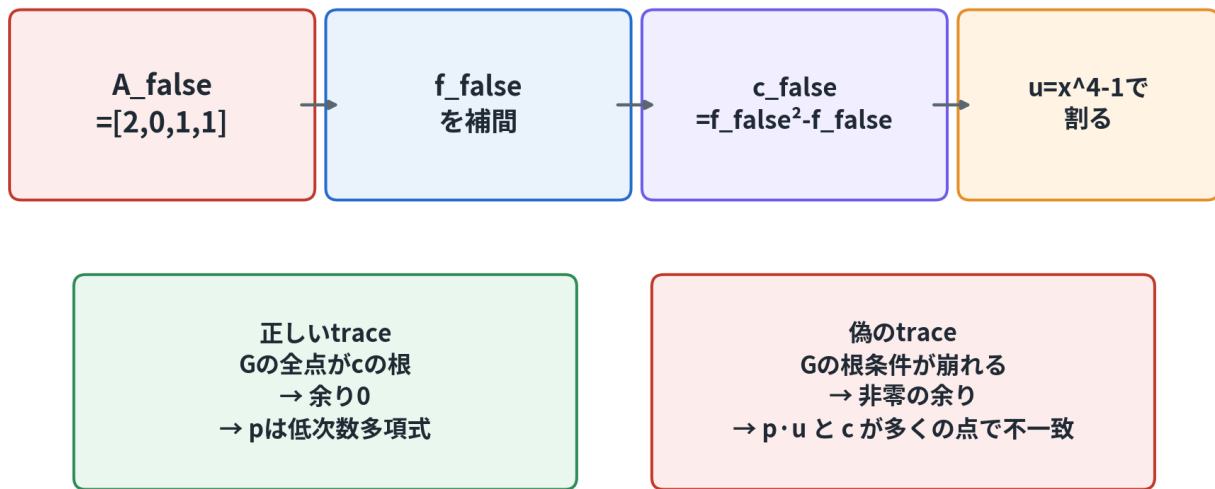
第10章 偽の trace を使うと何が起きるか

原論文参照：pp. 34-35

10.1 $A_{\text{false}}=[2,0,1,1]$

A_{false} の先頭要素 2 は boolean 制約を満たさない。証明者がこの trace から f_{false} を補間し、 $c_{\text{false}}=f_{\text{false}}^2-f_{\text{false}}$ を作ると、 c_{false} の根集合は G と一致しない。したがって $u=x^4-1$ で割ったときに非零の余りが生じる。

偽の trace はどこで破綻するか



検出経路：整合性queryで等式不一致、または不正な高次数応答をFRIで拒否。

図 17 偽の trace から作った制約多項式は u で正しく割り切れない

証明者が商の多項式部分だけを p_{false} としてコミットしても、 $p_{\text{false}}(x)u(x)$ と $c_{\text{false}}(x)$ は別の次数 6 多項式になる。両者が一致できる点は限られるため、 L が十分大きく、query を反復すれば高確率で不一致を検出できる。

原論文 p.25 の表記について

原論文は高次数攻撃の説明中に「 $g^4=16$ 」と記すが、前段の $g=4$ という定義では $4^2=16 \pmod{17}$ である。本書では攻撃の本質を保ち、「 G 上の一つの trace 位置」と表現する。

原論文の一部表現の読み方

p.21 の説明には「 f が G の全点で roots を持つ」と読める箇所があるが、正確には G の全点で 0 になるのは $c=f^2-f$ である。本書では式(5)の関係に従って整理した。

第 11 章 証明者と検証者の処理を工程別に整理する

原論文参照：pp. 35-36

証明者と検証者の責任分界

フェーズ	証明者 P	検証者 V
算術化	traceからf,c,pを構成	degree boundと制約式を把握
commitment	全評価表をMerkle木へ封印	rootを保持し、後のopeningと照合
整合性query	指定点の値とpathを返す	$p \cdot u = f^2 - f$ を自分で再計算
FRI commit	各fold層を計算しrootを送る	各root後にランダム α を選ぶ
FRI query	対点値とpathを返す	folding式を経路ごとに再計算
判定	証明データを完成	一つでも不一致なら拒否

VはPの計算を全面的に再実行しない。Pが先に固定したデータの「少数の局所関係」を再計算する。

図 18 工程ごとの証明者・検証者の責任分界

表 3 検証ロジックと拒否理由

工程	証明者が行うこと	検証者が行うこと	失敗時の意味
問題設定	秘密 trace を準備する	statement・制約・設定を固定する	別の命題を証明している
補間/LDE	f と L 上の全評価表を作る	degree bound を把握する	低次数符号化が成立しない
制約商	c と p を作る	query 点で $p \cdot u = f^2 - f$ を再計算	trace と制約が不整合
Merkle	評価表を root へ封印する	opening から root を再構成する	値が事前 commit 済みでない
FRI commit	各 fold 層を全計算し root 化する	root 後に α を選ぶ	challenge 依存の作り直し
FRI query	対点値と path を返す	folding を最終定数まで追う	低次数性が確認できない
判定	証明を提出する	全検査の AND を取る	一つでも失敗なら拒否

主要式と「誰が使うか」の対応

式	証明者 P	検証者 V
$f(g^i) = A_i$	P: traceからfを構成	V: 直接traceは見ない
$c(x) = f(x)^2 - f(x)$	P: boolean制約を多項式化	V: query点で右辺を再計算
$u(x) = x^N - 1$	P: 商を作る	V: 公開Nとzから自分で計算
$p(x) = c(x)/u(x)$	P: composition polynomialを構成	V: 低次数性をFRIで検査
$p(z)u(z) = f(z)^2 - f(z)$	P: openingを返す	V: 整合性を直接判定
$p_k = g_k(x^2) + xh_k(x^2)$	P: even/odd分解	V: 対点値から g_k, h_k を復元
$p_{k+1} = g_k + \alpha_k h_k$	P: 次FRI層を作る	V: query経路で同じ値を再計算

図 19 主要式と証明者・検証者の対応

11.1 検証者は何を「自分で」計算するのか

- 公開 N と query 点 z から $u(z) = z^N - 1$ を計算する。
- 受け取った $f(z)$ から $c(z) = f(z)^2 - f(z)$ を計算する。
- $p(z)u(z)$ と $c(z)$ を比較する。
- Merkle path を hash し直して root へ到達するか確認する。
- FRI 対点値から even/odd 値を復元し、次層の期待値を計算する。
- 最終的な期待値と、証明者が先に送った定数を比較する。

11.2 検証者が再計算しないもの

- 秘密 trace A 全体の再実行・再構成。
- L 上の f 、 p 、FRI 各層の全評価値。
- Merkle 木全体。必要な兄弟 hash だけを受け取る。
- 補間多項式の全係数。少数の opening と低次数証明で代替する。

第 12 章 対話型プロトコルを非対話型証明として読む

原論文参照：pp. 2, 28 footnote

原論文では、検証者が z や α を対話的に送る。実際の proof object では、Fiat-Shamir 変換により、それまでの transcript をハッシュして challenge を生成できる。検証者は証明受領後に同じ byte 列を同じ順序でハッシュし、同じ challenge を再生成する。

対話型から非対話型へ：Fiat-Shamirの置換

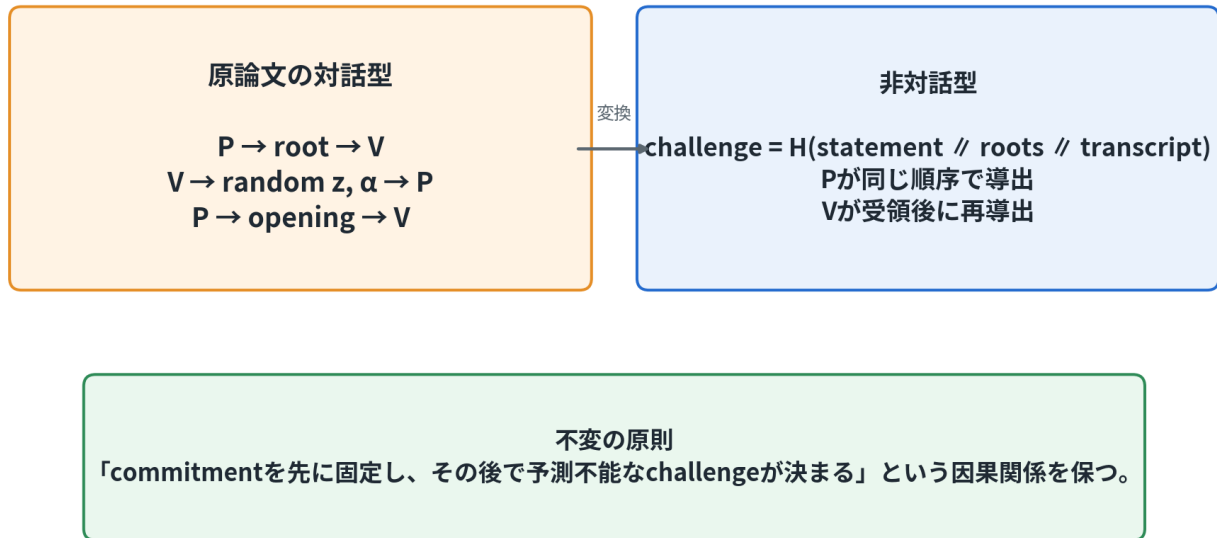


図 20 対話型 challenge を Fiat-Shamir で置き換える

12.1 何が変わり、何が変わらないか

表 4 対話型と非対話型の対応

項目	対話型	非対話型
challenge の生成	V が乱数を送る	$H(\text{statement} \parallel \text{commitments} \parallel \text{transcript})$
証明者の出力	複数往復の応答	一つの proof object
検証者の処理	送った乱数を使用	transcript を再生して challenge を再導出
重要な因果関係	root 送付後に乱数	root を hash へ吸収後に challenge

実装上の要点

field element の符号化、root の順序、domain separation、公開 statement の結合が一つでも異なると、証明者と検証者の challenge が一致しない。

第13章 原論文の簡略化と現代的 ZK-STARK への接続

原論文参照：pp. 2, 12 footnote, 14 footnote, 21-22 footnote

原論文は、理解しやすさを優先して、効率と一般性を意図的に削っている。そのため、本例をそのまま実用 STARK の仕様とみなすべきではない。ただし、trace の多項式化、制約商、commitment、ランダム query、FRI という因果関係は、現代的 STARK を理解する有効な骨格である。

原論文の教育用モデルと現代的 STARK の対応

論点	原論文のモデル	現代的実装での一般的姿
制約	単一のboolean制約	複数列・境界・遷移制約をランダム合成
ZK	queryを $L \setminus G$ に限定	トレース多項式をランダムマスク
整合性検査	fとpの点検査	composition + OODS/DEEPで束ねる
FRI	古典的binary folding	arity・拡大体・DEEP-FRI等を採用し得る
非対話化	本文は対話型	Fiat-Shamirでproof object化
効率	$L=F^*$ の小例	FFT-friendlyなcosetと実用パラメータ

本解説書は原論文の流れを忠実に説明し、相違点は「補足」として明示する。

図 21 原論文の教育用モデルと現代的 STARK の一般的対応

13.1 ゼロ知識について

原論文は、検証者の query を $L \setminus G$ へ限定することで、 $f(g^i) = A_i$ を直接開かない構成を採る。しかし、原論文の脚注が認めるとおり、実用的な ZK-STARK では通常、commit された値を照会しても秘密が漏れないよう、trace polynomial にランダムマスキングを施す。したがって、本例の query 制限はゼロ知識の直観を示す簡略化として読むべきである。

13.2 composition polynomial について

本例では一つの制約 $c = f^2 - f$ しかないため、 $p = c/u$ を composition polynomial と呼ぶ。実際の AIR では、複数列の境界・遷移・lookup・permutation 等から多数の商が生じ、それらを検証者由来のランダム係数で束ねる。

13.3 OODS/DEEP が登場しない点

原論文は、 f と p の L 上の評価表へ直接コミットし、同一の query 点で式(10)を点検査する。現代的構成では、評価領域外の点における opening を使い、composition と trace、低次数検査をより強く結び付ける OODS/DEEP 系の技法が用いられることがある。本書の説明は原論文に従うため、OODS/DEEP をプロトコル本体へ追加していない。

この論文から持ち帰るべき骨格

「計算を制約付き低次数多項式へ変換する」「先に commit する」「ランダムな少数点で制約を確認する」「FRI で低次数性を確認する」という四段構造である。

第14章 一枚で振り返る検証者の判定プロセス

検証者の最終判定は、複数の独立検査の論理積である。どれか一つが成功すればよいのではなく、すべてが成功しなければならない。

表5 検証者の受理・拒否判定

検査	Vの入力	Vの再計算	拒否条件
statement binding	N, F, G, L , 制約	設定の整合	未対応の設定
Merkle f	$f(z)$, path	root 再構成	$MT_f(\text{root})$ と不一致
Merkle p	$p(z)$, path	root 再構成	$MT_p(\text{root})$ と不一致
constraint relation	$f(z), p(z), z$	$u(z), f^2 - f, p \cdot u$	等式不一致
FRI openings	各層の対点値, path	root と folding 式	層間不一致
final value	最終定数	経路の最終評価	定数不一致

受理の意味

Vが受理するのは、「固定された公開設定に対し、commitされた f と p が制約関係を満たし、 p が所定の低次数構造を持つ」という主張である。現実世界の入力の真正性や、アプリケーション仕様の妥当性は別途保証が必要である。

Appendix A 数値例の主要データ

A.1 公開設定と trace

表 6 数値例の基本設定

項目	値
有限体	F_{1_7}
N	4
generator g	4
G	{1,4,13,16}
L	$F_{1_7} \setminus \{0\}$
A_{true}	[1,0,1,1]
A_{false}	[2,0,1,1]

A.2 多項式

$$f(x)=16x^3+13x^2+x+5$$

$$c(x)=x^6+8x^5-3x^4-x^2-8x+3$$

$$u(x)=x^4-1$$

$$p(x)=x^2+8x-3$$

A.3 L 上の評価値

表 7 f と p の評価表 (4 点ずつ表示)

評価点	$f(x)$	$p(x)$
$x=[1, 2, 3, 4]$	$f=[1, 0, 13, 0]$	$p=[6, 0, 13, 11]$
$x=[5, 6, 7, 8]$	$f=[6, 8, 0, 10]$	$p=[11, 13, 0, 6]$
$x=[9, 10, 11, 12]$	$f=[15, 9, 3, 8]$	$p=[14, 7, 2, 16]$
$x=[13, 14, 15, 16]$	$f=[1, 10, 12, 1]$	$p=[15, 16, 2, 7]$

A.4 FRI の数値

表 8 FRI commitment/query の対応

段階	多項式	challenge	結果
Layer 0	$p_0(x)=x^2+8x-3$	$\alpha_0=10$	$p_1(y)=y+9$

段階	多項式	challenge	結果
Layer 1	$p_1(y)=y+9$	$\alpha_1=15$	$p_2=7$
Query 1	$z=12, -z=5$	$p(12)=16, p(5)=11$	$p_1(8)=0$
Query 2	$8, -8=9$	$p_1(8)=0, p_1(9)=1$	$p_2=7$

Appendix B 式から処理を逆引きする

主要式と「誰が使うか」の対応

式	証明者 P	検証者 V
$f(g^i)=A_i$	P: traceからfを構成	V: 直接traceは見ない
$c(x)=f(x)^2-f(x)$	P: boolean制約を多項式化	V: query点で右辺を再計算
$u(x)=x^N-1$	P: 商を作る	V: 公開Nとzから自分で計算
$p(x)=c(x)/u(x)$	P: composition polynomialを構成	V: 低次数性をFRIで検査
$p(z)u(z)=f(z)^2-f(z)$	P: openingを返す	V: 整合性を直接判定
$p_k=g_k(x^2)+xh_k(x^2)$	P: even/odd分解	V: 対点値から g_k, h_k を復元
$p_{k+1}=g_k+\alpha_k h_k$	P: 次FRI層を作る	V: query経路で同じ値を再計算

図 22 主要式と処理主体の対応

表 9 主要式の逆引き表

式	役割	主な検査
$f(g^i)=A_i$	trace の符号化	後続の commitment と制約で間接検査
$c=f^2-f$	boolean 制約の代数化	V が $f(z)$ から再計算
$u=x^N-1$	G の消失多項式	V が公開 N, z から再計算
$p=c/u$	制約が G 全点で成立することの商表現	整合性 query + FRI
$p \cdot u=f^2-f$	f と p の結合	ランダム z で点検査
$p_{k+1}=g_k+\alpha_k h_k$	FRI folding	層間 query で再計算

Appendix C 用語集

表 10 本書の主要用語

用語	本書での意味
CI statement	計算結果または trace が公開された計算規則を満たすという主張。
trace	証明者が保持する秘密列 A。一般 STARK では実行状態の表。
trace polynomial f	G 上で trace を評価値として持つ低次数多項式。
constraint polynomial c	本例では $f^2 - f$ 。制約違反を多項式として表す。
zerofier u	所定の制約点 G ですべて 0 になる x^{N-1} 。
composition polynomial p	本例では c/u 。一般には複数制約商のランダム合成。
Merkle commitment	評価表全体を後から変更できないよう root へ封印する。
opening	query 位置の値と Merkle path を提示すること。
FRI	評価表が低次数 Reed-Solomon 符号に近いことを少数 query で検査するプロトコル。
commitment round	P が FRI 層を生成し、各層の root を先に送る段階。
querying round	V がランダム経路で folding 式を再計算する段階。
Fiat-Shamir	対話的 challenge を transcript hash から導出し非対話化する技法。

参考文献

- Berentsen, A., Lenzi, J., and Nyffenegger, R. (2022). A Walk-Through of a Simple zk-STARK Proof. SSRN 4308637.
(本書の主要対象)
- Ben-Sasson, E., Bentov, I., Horesh, Y., and Riabzev, M. (2017). Fast Reed-Solomon Interactive Oracle Proofs of Proximity.
- Ben-Sasson, E., Bentov, I., Horesh, Y., and Riabzev, M. (2018). Scalable, transparent, and post-quantum secure computational integrity. IACR ePrint 2018/046.
- Buterin, V. (2017-2018). STARKs, Part I-III.
- StarkWare (2019-2020). STARK Math / Arithmetization / Low Degree Testing / STARK 101 series.

出典と補足の区分

本文のプロトコル、式、数値例、構成は原論文に基づく。原論文の簡略化、現代的 STARK との相違、数値・表記上の注意は、本書独自の補足として明示した。